

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ ІМЕНІ МИХАЙЛА
ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ
ІНЖЕНЕРІЇ ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«АЛГОРИТМИ І СТРУКТУРА ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 5

виконала:
студентка групи КН-25-1
Бойко Д.О.
Перевірив:
Старший викладач кафедри АІС
Сидоренко В.М.

Кременчук 2026

Практична робота № 5

Тема. Графи. Ациклічні графи

Мета:набути практичних навичок розв'язання задач топографічного сортування та оцінювання їх асимптотичної складності.

Постановка завдання:

Виконати індивідуальне завдання. Завдання полягає у розв'язанні задачі, яку потрібно вибрати зі списку, наведеного нижче. Номер варіанта відповідає номеру студента у списку групи.

Завдання 1

1. Задано ациклічний граф: $\{1,2,3,4,5,6,7\}\{(1,2),(1,3),(1,6),(3,7), (1,4), (4,5)\}$. Побудувати граф і розв'язати задачу топологічного сортування за допомогою алгоритму Кана.

Розв'язання

Побудова графа

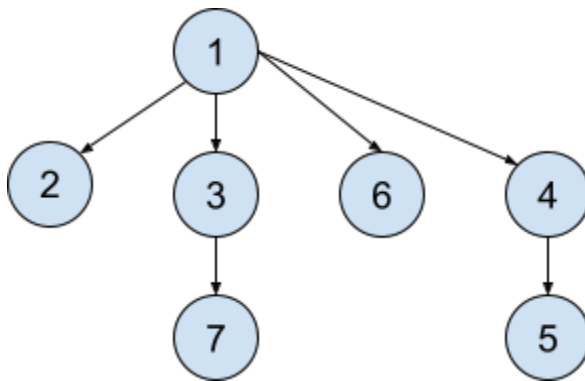


Рисунок 1-Орієнтований ациклічний граф

Алгоритм Кана

Крок 1

Знаходимо вершини без вхідних ребер.

Вхідні степені вершин:

Вершина	Вхідний степінь
1	0
2	1
3	1
4	1
5	1
6	1
7	1

Початкова множина:

$$S = \{1\}$$

Результат:

$$L = [\]$$

Крок 2

Беремо вершину 1.

Додаємо її у список результату:

$$L = [1]$$

Видаляємо ребра:

- (1, 2)
- (1, 3)
- (1, 4)
- (1, 6)

Тепер вершини 2,3,4,6 не мають вхідних ребер.

$$S = \{2, 3, 4, 6\}$$

Крок 3

Беремо вершину 2.

$$L = [1, 2]$$

$$S = \{3, 4, 6\}$$

Крок 4

Беремо вершину 3.

$$L = [1, 2, 3]$$

Видаляємо ребро:

- $(3, 7)$

Тепер вершина 7 не має вхідних ребер.

$$S = \{4, 6, 7\}$$

Крок 5

Беремо вершину 4.

$$L = [1, 2, 3, 4]$$

Видаляємо ребро:

- $(4, 5)$

Тепер вершина 5 не має вхідних ребер.

$$S = \{5, 6, 7\}$$

Крок 6

Беремо вершину 5.

$$L = [1, 2, 3, 4, 5]$$

$$S = \{6, 7\}$$

Крок 7

Беремо вершину 6.

$$L = [1, 2, 3, 4, 5, 6]$$

$$S = \{7\}$$

Крок 8

Беремо вершину 7.

$L = [1, 2, 3, 4, 5, 6, 7]$

$S = \emptyset$

Результат топологічного сортування

$[1, 2, 3, 4, 5, 6, 7]$

Програма мовою Python

```
1  from collections import defaultdict, deque
2
3  graph = {
4      1: [2, 3, 6, 4],
5      2: [],
6      3: [7],
7      4: [5],
8      5: [],
9      6: [],
10     7: []
11 }
12
13 in_degree = defaultdict(int)
14
15 for vertex in graph:
16     for neighbor in graph[vertex]:
17         in_degree[neighbor] += 1
18
19 queue = deque()
20
21 for vertex in graph:
22     if in_degree[vertex] == 0:
23         queue.append(vertex)
24
25 result = []
```

```
27  while queue:
28     vertex = queue.popleft()
29     result.append(vertex)
30
31     for neighbor in graph[vertex]:
32         in_degree[neighbor] -= 1
33
34         if in_degree[neighbor] == 0:
35             queue.append(neighbor)
36
37 print("Топологічне сортування:")
38 print(result)
```

Топологічне сортування – це впорядкування вершин орієнтованого ациклічного графа таким чином, що для кожного ребра (u, v) вершина u розташована перед вершиною v .

Висновок

У ході виконання практичної роботи було побудовано орієнтований ациклічний граф та виконано його топологічне сортування за допомогою алгоритму Кана. У результаті отримано послідовність [1, 2, 3, 4, 5, 6, 7].

Алгоритм Кана дозволяє ефективно виконувати топологічне сортування ациклічних графів.

Контрольні питання

1. Які переваги і недоліки алгоритму Кана порівняно з алгоритмом DFS для топологічного сортування графа?

Алгоритм Кана є простішим для розуміння та дозволяє легко визначити вершини без вхідних ребер. Також він може виявляти цикли у графі. Недоліком є необхідність зберігання вхідних степенів вершин.

Алгоритм DFS працює швидко та використовує обхід у глибину, але є складнішим для розуміння.

2. Яка складність часу і пам'яті для кожного з алгоритмів у найгіршому і найкращому випадках?

Для алгоритму Кана:

- часова складність:
 $O(V + E)$
- просторова складність
 $O(V)$

Для алгоритму DFS:

- часова складність:
 $O(V + E)$
- просторова складність:
 $O(V)$

Де:

- V – кількість вершин,

- E — кількість ребер.

3. Чи можна застосовувати алгоритм Кана до графів з вагами на ребрах? Як це порівняти з DFS?

Так, алгоритм Кана можна застосовувати до графів з вагами, оскільки ваги ребер не впливають на порядок топологічного сортування. Аналогічно працює і алгоритм DFS.

4. Як впливає структура графа на швидкість роботи кожного з цих алгоритмів?

Чим більше вершин і ребер у графі, тим більше часу на виконання алгоритмів. Для розріджених графів алгоритми працюють швидше, ніж для щільних графів.

5. Чи є обмеження використання кожного алгоритму для певних типів графів або завдань?

Обидва алгоритми застосовуються лише для орієнтованих ациклічних графів. Якщо граф містить цикл, топологічне сортування неможливе.

6. Які варіанти оптимізації можна застосувати для кожного алгоритму з метою поліпшення його продуктивності?

Для покращення продуктивності можна:

- використовувати ефективні структури даних;
- застосовувати списки суміжності;
- зменшити кількість повторних обходів графа;
- Оптимізувати роботу з чергою та рекурсією.